

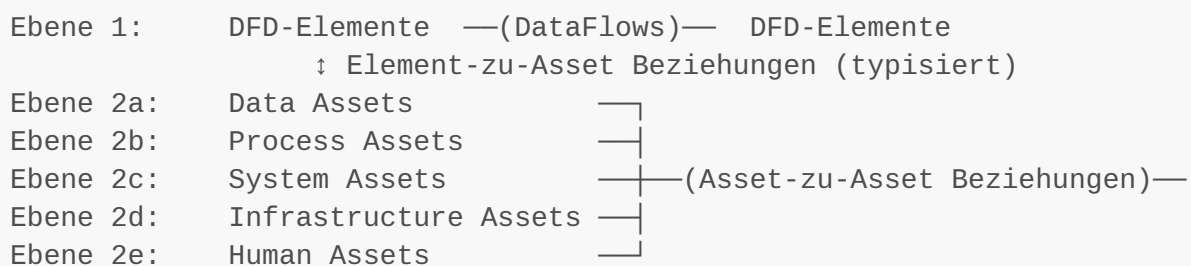
TARAFLOW: Asset-zu-Asset Beziehungen

© Jürgen Messerer · 2026 · Alle Rechte vorbehalten

Zweck dieses Dokuments: Formale Definition der Asset-zu-Asset Beziehungen in TARAFLOW. Diese Beziehungen ergänzen die Element-zu-Asset Beziehungen und ermöglichen die Modellierung direkter semantischer Verbindungen zwischen Assets – ohne Umweg über DFD-Elemente.

1. Konzept und Einordnung im Graph

Der TARAFLOW-Graph hat zwei Hauptebenen – wobei Ebene 2 aus fünf Teilebenen besteht, eine pro Asset-Kategorie:



Ebene 2 ist ein **heterogener Graph** mit fünf Knotentypen. Die erlaubten Beziehungstypen sind abhängig von der Kombination **Quell-Kategorie × Ziel-Kategorie**:

Innerhalb einer Kategorie: Data→Data, Process→Process, ...
Zwischen Kategorien: Data→Process, Process→Human,
System→Infrastructure, ...

Das entspricht dem gleichen Prinzip wie Ebene 1: auch DFD-Elemente haben verschiedene Typen (Process, DataStore, ExternalEntity, ...) mit typenabhängigen Verbindungsregeln.

Asset-zu-Asset Beziehungen beschreiben direkte semantische Verbindungen zwischen Assets die sich nicht natürlich über ein DFD-Element ausdrücken lassen.

Beispiel:

Data Asset "Admin Audit Logs"
└ affects_privacy → Human Asset "System Admin Role"

→ Diese Beziehung existiert unabhängig davon welches DFD-Element die Audit Logs erzeugt oder liest.

2. Zwei-Stufen-Regelwerk

TARAflow unterscheidet zwei Stufen von Asset-zu-Asset Beziehungen:

Stufe 1: Core Rules

- generisch, domänenunabhängig
- fest eingebaut ins Tool
- analytisch wirksam (STRIDE, Safety, Kritikalität)

Stufe 2: Domain Extensions

- domänenspezifisch, konfigurierbar
- in Phase 0 (Projektprofil) aktivierbar oder manuell definierbar
- default: nur dokumentarisch
- optional: analytisch wirksam (mit Pflicht-Rationale)

3. Stufe 1: Core Rules (generisch)

3.1 Innerhalb derselben Asset-Kategorie

Data → Data:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
<i>derives_from</i>	Asset B wird aus Asset A abgeleitet	Tampering (A beeinflusst B)	transitiv
<i>aggregates</i>	Asset B aggregiert mehrere A-Instanzen	Tampering, Information Disclosure	transitiv
<i>supersedes</i>	Asset B ersetzt Asset A (z.B. neue Firmware)	Tampering, Repudiation	transitiv

Process → Process:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
<i>triggers</i>	Prozess A löst Prozess B aus	Tampering, Spoofing	direct wenn B safety-kritisch
<i>depends_on</i>	Prozess A benötigt Prozess B	Denial of Service	transitiv
<i>suspends</i>	Prozess A unterbricht Prozess B	Denial of Service	direct wenn B safety-kritisch

System → System:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
<i>depends_on</i>	System A benötigt System B	Denial of Service	transitiv
<i>integrates</i>	System A integriert System B	Tampering, Spoofing	transitiv

Infrastructure → Infrastructure:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
powers	Infra A versorgt Infra B mit Strom	Denial of Service	indirect
houses	Infra A beherbergt Infra B physisch	Tampering	indirect

Human → Human:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
manages	Person A verwaltet Person B	Elevation of Privilege	–
reports_to	Person A rapportiert an Person B	–	–

3.2 Zwischen verschiedenen Asset-Kategorien

Data → Process:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
required_by	Daten werden für Prozessausführung benötigt	Tampering, DoS	transitiv via Prozess
consumed_by	Daten werden durch Prozess transformiert	Tampering	transitiv

Data → Human:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
affects_privacy	Daten enthalten personenbezogene Informationen	Information Disclosure	–
exposes	Daten exponieren Person gegenüber Risiken	Information Disclosure	indirect

Process → System:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
runs_on	Prozess läuft auf System	Tampering, Elevation of Privilege	transitiv
requires	Prozess benötigt System	Denial of Service	transitiv

Process → Human:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
affects_safety	Prozess kann Menschen physisch gefährden	Tampering, DoS	direct

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
affects_privacy	Prozess verarbeitet personenbezogene Daten	Information Disclosure	–
operated_by	Prozess wird von Mensch bedient	Spoofing, Repudiation	indirect

System → Infrastructure:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
hosted_on	System läuft auf Infrastruktur	Tampering	indirect
powered_by	System wird von Infrastruktur versorgt	Denial of Service	indirect

Human → Process:

Beziehung	Semantik	STRIDE-Relevanz	Safety-Relevanz
responsible_for	Person ist verantwortlich für Prozess	Repudiation, Accountability	–
authorized_for	Person ist autorisiert Prozess auszuführen	Spoofing, Elevation of Privilege	–

3.3 Analytische Wirkung der Core Rules

Core Rules sind **analytisch wirksam** – sie beeinflussen:

STRIDE-Ableitung:

Auslöser	→ Pflicht-Threat
Data A derives_from Data B	→ Tampering auf B impliziert
Tampering auf A	
Process A triggers Process B[safety]	→ DoS auf A = Safety-relevanter Threat
Process A affects_safety Human	→ Tampering + DoS als Pflicht-Threats
Data affects_privacy Human	→ Information Disclosure als Pflicht-Threat

Safety-Propagation:

Process A triggers Process B[direct, fatality]
→ Process A erhält relevance: 'indirect' automatisch (Hop 1)

```
Data required_by Process[direct, fatality]
→ Data erhält relevance: 'indirect' automatisch (Hop 1)
```

Kritikalitäts-Ableitung:

```
Folgt dem bestehenden Derived/Manual Pattern:
→ Core-Rule-Ableitungen = source: "derived"
→ keine zusätzliche Dokumentationspflicht
→ im Audit-Report automatisch als "systemisch abgeleitet" markiert
```

4. Stufe 2: Domain Extensions

4.1 Konzept

Domain Extensions sind **domänenspezifische Beziehungen** die der Analyst selbst definiert oder aus einem vordefinierten Domain-Katalog aktiviert.

Default-Verhalten: dokumentarisch

```
Domain-Beziehung "administered_to":
→ erscheint in Dokumentation und Asset-Inventar
→ beeinflusst STRIDE-Analyse NICHT
→ beeinflusst Safety-Propagation NICHT
→ kein Einfluss auf Kritikalität
```

Optional: analytisch wirksam

```
Domain-Beziehung "administered_to" (analytisch aktiviert):
→ Analyst markiert Beziehung als "analytisch wirksam"
→ Pflicht-Rationale: z.B. "impliziert affects_safety auf Human Asset"
→ Pflicht-STRIDE-Mapping: welche Kategorien werden abgeleitet?
→ Source: "manual" → verbatim im Audit-Report dokumentiert
```

4.2 Vordefinierte Domain-Kataloge

Medical (IEC 81001-5-1, ISO 14971):

Beziehung	Von → Nach	Default	Analytisch wirksam als
<code>administered_to</code>	Process → Human	dokumentarisch	<code>affects_safety</code>
<code>treats</code>	Process → Human	dokumentarisch	<code>affects_safety</code>
<code>diagnoses</code>	Data → Human	dokumentarisch	<code>affects_privacy</code>
<code>prescribes_for</code>	Data → Human	dokumentarisch	<code>affects_privacy</code>

Beziehung	Von → Nach	Default	Analytisch wirksam als
<code>contraindicated_with</code>	Data → Data	dokumentarisch	<code>required_by</code>

Automotive (ISO 21434):

Beziehung	Von → Nach	Default	Analytisch wirksam als
<code>transports</code>	System → Human	dokumentarisch	<code>affects_safety</code>
<code>endangers</code>	System → Human	dokumentarisch	<code>affects_safety</code>
<code>controls_vehicle</code>	Process → System	dokumentarisch	<code>runs_on</code>

OT / Industrial (IEC 62443):

Beziehung	Von → Nach	Default	Analytisch wirksam als
<code>operates_near</code>	Process → Human	dokumentarisch	<code>affects_safety</code>
<code>exposes_to_hazard</code>	Infrastructure → Human	dokumentarisch	<code>affects_safety</code>
<code>physically_contains</code>	Infrastructure → Human	dokumentarisch	–
<code>controls_actuator</code>	Process → System	dokumentarisch	<code>runs_on</code>

Generic IT:

Beziehung	Von → Nach	Default	Analytisch wirksam als
<code>authenticates</code>	System → Human	dokumentarisch	<code>authorized_for</code>
<code>audits</code>	Process → Data	dokumentarisch	<code>responsible_for</code>
<code>encrypts</code>	System → Data	dokumentarisch	–
<code>backs_up</code>	Process → Data	dokumentarisch	<code>required_by</code>

4.3 Benutzerdefinierte Beziehungen

Der Analyst kann eigene Beziehungstypen definieren:

Benutzerdefinierte Beziehung:

```

├─ name:          z.B. "calibrates"
├─ von:           Asset-Kategorie (z.B. Process)
├─ nach:          Asset-Kategorie (z.B. System)
├─ semantik:      Freitextbeschreibung
├─ analytisch:    true / false
├─ wenn analytisch:
│   ├─ stride_mapping: [ "Tampering", "Repudiation" ]
│   ├─ safety_relevance: "direct" | "indirect" | "none"
│   └─ rationale:     Pflichtfeld (Freitext)
└─ domain:        z.B. "Medical", "Custom"

```

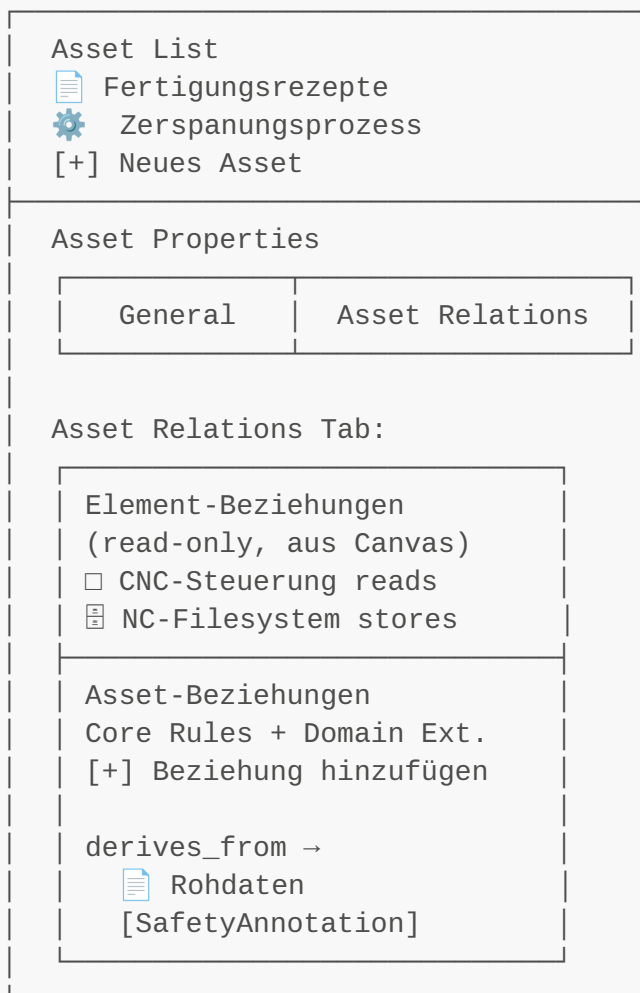
Validation-Regel:

```
IF analytisch === true AND rationale === ""  
THEN Validierungswarnung:  
    "Rationale für analytisch wirksame Domain-Beziehung ist Pflicht  
    (IEC 62443-4-1 Traceability)"
```

5. UI-Integration im DFD-Tab

Asset-zu-Asset Beziehungen werden im **Asset Description Side Panel** verwaltet – analog zum Element Description Panel mit zwei Tabs (General / Asset Relations):

Side Panel (overlay, vertical split):



Beziehung hinzufügen – Workflow:

1. [+] Beziehung hinzufügen
2. Beziehungstyp wählen
 - Core Rules (immer verfügbar)
 - Domain Extensions (falls in Phase 0 aktiviert)
 - Benutzerdefiniert

3. Ziel-Asset auswählen (aus Asset-Liste)
4. Optional: SafetyAnnotation setzen
5. Wenn Domain-Extension analytisch:
 - Rationale eingeben (Pflicht)

6. Auswirkung auf die Gesamtanalyse

6.1 Übersicht

	Core Rules	Domain (dokumentarisch)	Domain (analytisch)
STRIDE-Ableitung	✓ automatisch	✗	✓ manuell definiert
Safety-Propagation	✓ Hop-Logik	✗	✓ gemäss safety_relevance
Kritikalität	✓ derived	✗	✓ manual + Rationale
Dokumentation	✓ automatisch	✓ Asset-Inventar	✓ Rationale verbatim

6.2 Traceability-Kette

Massnahme

- Risk
- Threat (STRIDE-Kategorie)
- Asset-zu-Asset Beziehung (Core Rule oder Domain Extension)
- Asset
- DFD-Element

Vollständig rückverfolgbar – auch wenn die Threat-Ableitung über eine Asset-zu-Asset Beziehung erfolgte.

7. Abgrenzung zu Element-zu-Asset Beziehungen

Merkmal	Element-zu-Asset	Asset-zu-Asset
Modellierungsort	DFD-Canvas	Asset Properties Side Panel
Richtung	DFD-Element → Asset	Asset → Asset
Visualisierung	Canvas-Kante	Side Panel Liste
Analytisch wirksam	immer	Core: immer / Domain: optional
SafetyAnnotation	✓	✓
Derived/Manual Pattern	✓	✓

8. Beispiele

8.1 CNC-System (OT-Domain)

Process Asset "Zerspanungsprozess"

- └ depends_on (Core) → Process Asset "Not-Halt-Prozess"
 - Pflicht-Threat: DoS auf Not-Halt = DoS auf Zerspanungsprozess
 - Safety: indirect (Not-Halt ist direct, Zerspanungsprozess erbt indirect)

Data Asset "Fertigungsrezepte"

- └ required_by (Core) → Process Asset "Zerspanungsprozess"
 - Pflicht-Threat: Tampering auf Rezepte = Tampering auf Zerspanungsprozess
 - Safety: indirect (Hop 1 via Zerspanungsprozess)

8.2 Medical Device (Medical-Domain)

Process Asset "Bolus-Abgabe-Sequenz"

- └ administered_to (Domain, analytisch) → Human Asset "Patient"
 - Rationale: "Direkte Verabreichung impliziert affects_safety"
 - stride_mapping: ["Tampering", "Denial of Service"]
 - safety_relevance: "direct"
 - Pflicht-Threat: Tampering auf Bolus-Sequenz = CRITICAL (Safety Override)

Data Asset "Medikamenten-Grenzwerte"

- └ required_by (Core) → Process Asset "Medikamenten-Validierungsablauf"
 - Tampering auf Grenzwerte = Tampering auf Validierungsablauf
 - Safety: indirect (Validierungsablauf ist direct)